Lab 7. Packaging with Docker

In addition to helping with the process of building and testing software, Docker can also provide a standard packaging format with Docker images. This packaging format comes with the distribution mechanism, i.e., Docker image registries.

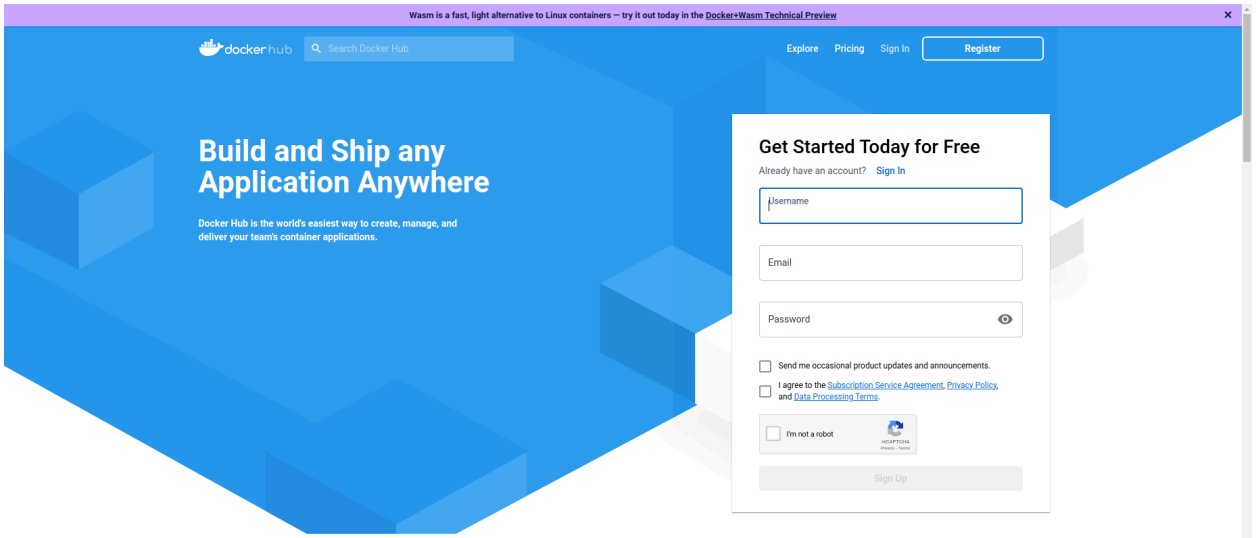
By the end of this lab exercise, you should be able to:

- Discuss Docker's standard image format
- Test and build Docker images manually
- Write Dockerfiles
- Understand the automated, iterative Docker image process
- Add the Docker packaging stage to the Jenkins pipeline
- Create per-stage agent configurations in Jenkinsfile

Registering with Docker Hub

As you prepare to work with the registry, it's important to have your own account. This account will enable you to build and push images to the registry. For the purpose of this tutorial, we'll be using a hosted registry called Docker Hub.

If you do not already have an account, visit the following link: https://hub.docker.com/ and sign up using your email.



Docker Hub is the world's largest

Check your email inbox and check the activation email sent by Docker.

After clicking on the activation link, you will be redirected to a login page. Enter your credentials and log in.

You will be taken to the Docker Hub main page. The registration process is complete and you have an account with Docker Hub.

Test Building Docker Image for the `worker` Application

Before you start building automated images, you will manually create a Docker image. You have already used the pre-built image from the registry in the last section. In this subsection, you will create an image with the **`worker`** application installed. Since **`worker`** a Java-based application that needs Maven to build, you will base your work on Maven's existing official image.

Enter the `example-voting-app/worker` directory.

Create a container named **`build`** with the **`maven:3.9.8-sapmachine-21`** image:

```
docker run -idt --name build maven:3.9.8-sapmachine-21 sh
```

Copy the source code. You must be inside the `example-voting-app/worker` directory for this to work:

```
docker container cp . build:/app
```

Connect to the container to compile and package the code:

```
docker exec -it build sh
```

```
cd app
```

```
mvn package
```

Verify the `.jar` file has been built:

```
ls target/
```

Run the `.jar` file with the following **java** command:

```
java -jar target/worker-jar-with-dependencies.jar
```

Sample output:

```
Waiting for redis
Waiting for redis
Waiting for redis
Waiting for redis
Waiting for redis
Waiting for redis
ctrl+c
```

*Use `Ctrl+c` to exit.

The above is the expected output. The `worker` app is waiting for `redis`.

Move the artifact:

```
mv target/worker-jar-with-dependencies.jar /run/worker.jar
```

Remove the source code:

```
rm -rf /app/*
```

Exit the container shell to return to your machine's shell:

```
exit
```

Commit the container to an image.

```
docker container commit build <xxxxx>/worker:v1
```

List your containers:

```
docker ps
```

Look for the container named **build** and take note of the container **ID**.

Commit the container into an image.

***NOTE**: Be sure to replace **<xxxxx>** with your own Docker Hub Account ID, that is your username.*

Test before pushing by launching the container with the packaged app:

```
docker run --rm -it <xxxxx>/worker:v1 java -jar /run/worker.jar
```

```
..
```

```
..
```

```
..
```

*Use **Ctrl + c** to exit.

Push the image to the registry.

***NOTE**: Before you push the image, you need to be logged in to the registry, with the Docker Hub ID created earlier.*

Login using the following command:

```
docker login
```

To push the image, first list it:

```
docker image ls
```

Sample output:

```
REPOSITORY          TAG    IMAGE ID        CREATED           SIZE
eeganlf/worker      v2     90cbeb6539df    18 minutes ago    194MB
eeganlf/worker      v1     c0199f782489    34 minutes ago    189MB
```

To push the image, use:

```
docker push <xxxxx>/worker:v1
```

Automated Image Build with Dockerfile

Now, let's build the same image, this time using a Dockerfile.

To do this, create a file and name it **Dockerfile** in the root of the source code for the **worker** app. It should have the path **example-voting-app/worker/Dockerfile**:

```
FROM maven:3.9.8-sapmachine-21
WORKDIR /app
COPY . .
RUN mvn package && \
    mv target/worker-jar-with-dependencies.jar /run/worker.jar && rm -rf /app/*
CMD ["java", "-jar", "/run/worker.jar"]
```

Let's build the image. Make sure you are in the `example-voting-app/worker` directory:

```
docker image build -t <xxxxx>/worker:v2 .
```

```
docker image ls
```

Try building again:

```
docker image build -t <xxxxx>/worker:v2 .
```

This time it runs much faster because Docker uses the cache.

Test the image using:

```
docker container run --rm -it <xxxxx>/worker:v2
```

You will see "waiting for redis" repeating in the console if it works.

*Use **`Ctrl + c`** to exit.

Tag the image as **`latest`**:

```
docker image tag <xxxxx>/worker:v2 <xxxxx>/worker:latest
```

```
docker image ls
```

Finally, publish images to the registry:

```
docker image push <xxxxx>/worker:latest
```

```
docker image push <xxxxx>/worker:v2
```

You must commit the Dockerfile into your Git repo before you can automate this process with Jenkins:

```
git add worker/Dockerfile
```

```
git commit -am "adding Dockerfile for worker"
```

```
git push origin feature/dockerfiles
```

Adding Docker Build and Publish Stages to Jenkinsfile

To have Jenkins build and publish a Docker image, you must add a pipeline script block. Copy and paste the following code inside the stages section between the **package** and **post** stages of your **worker/Jenkinsfile**:

```
 stage('docker-package'){
     agent any
     steps{
       echo 'Packaging worker app with docker'
       script{
         docker.withRegistry('https://index.docker.io/v1/',
'dockerlogin') {  def workerImage =
docker.build("xxxx/worker:v${env.BUILD_ID}", "./worker")
             workerImage.push()
             workerImage.push("latest")
         }
       }
     }

}
```

The entire **worker/Jenkinsfile** pipeline should appear as follows, BUT notice the red text which should be replaced with your Docker Hub ID:

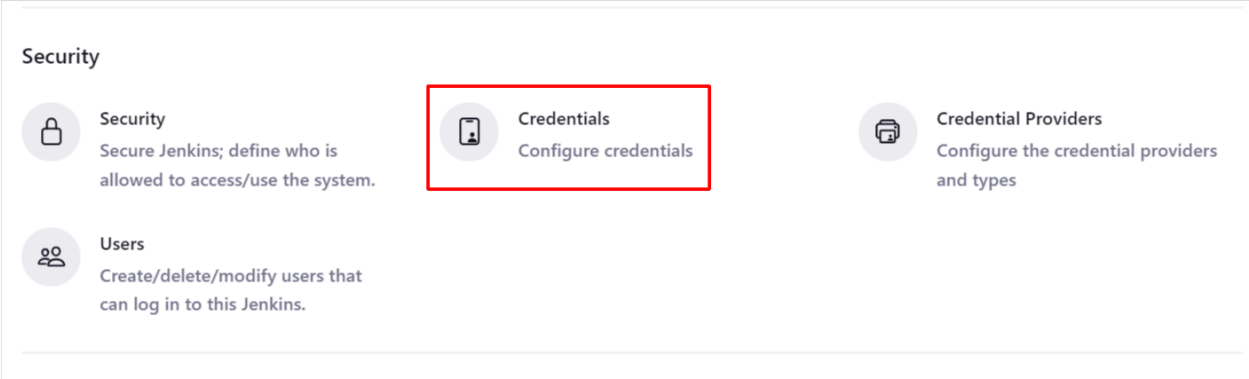
```
pipeline{

    agent{
      docker{
        image 'maven:3.9.8-sapmachine-21'
        args '-v $HOME/.m2:/root/.m2'
      }
    }

    stages{
        stage('build'){
            steps{
                echo 'building worker app'
                dir('worker'){
                  sh 'mvn compile'
                }
            }
        }
        stage('test'){
            steps{
                echo 'running unit tests on worker app'
```

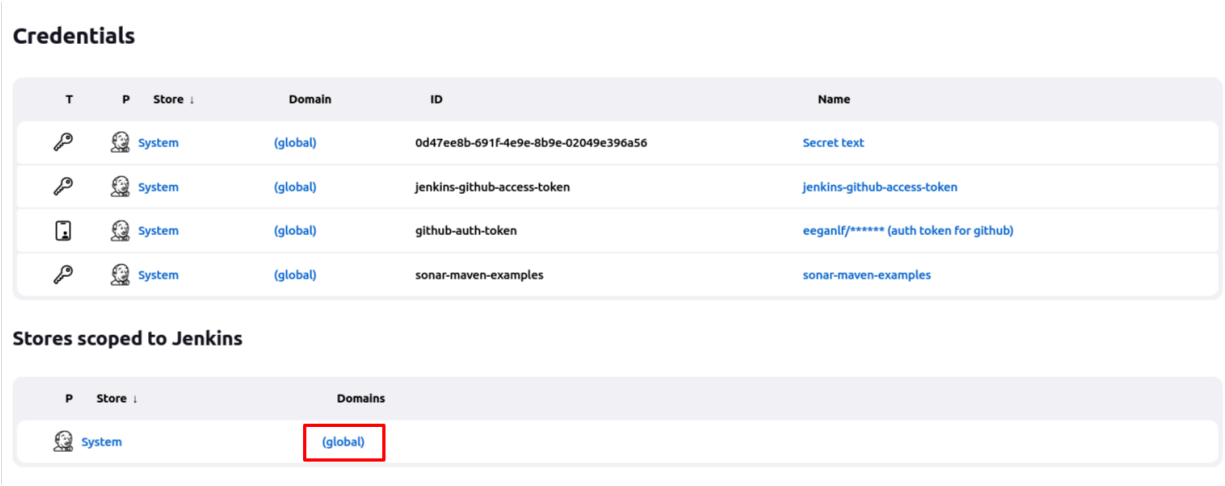
```
                dir('worker'){
                  sh 'mvn clean test'
                }
            }
        }
        stage('package'){
            steps{
                echo 'packaging worker app into a jar file'
                dir('worker'){
                  sh 'mvn package -DskipTests'
                  archiveArtifacts artifacts: '**/target/*.jar',
fingerprint:  true
                }
            }
        }
        stage('docker-package'){
            agent any
            steps{
              echo 'Packaging worker app with docker'
              script{
                docker.withRegistry('https://index.docker.io/v1/',
'dockerlogin') {
                    def workerImage = docker.build("xxxxx/worker:v$
{env.BUILD_ID}", "./worker")
                    workerImage.push()
                    workerImage.push("latest")
                }
              }
            }
        }
    }

    post{
        always{
            echo 'the job is complete'
        }

    }

}
```

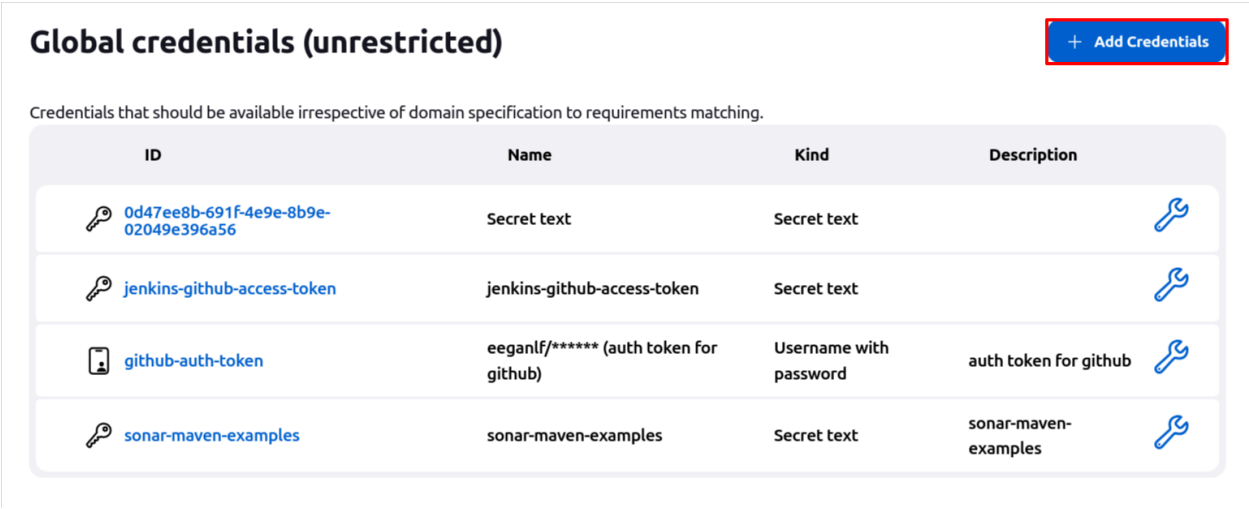
Adding Docker Hub Credentials

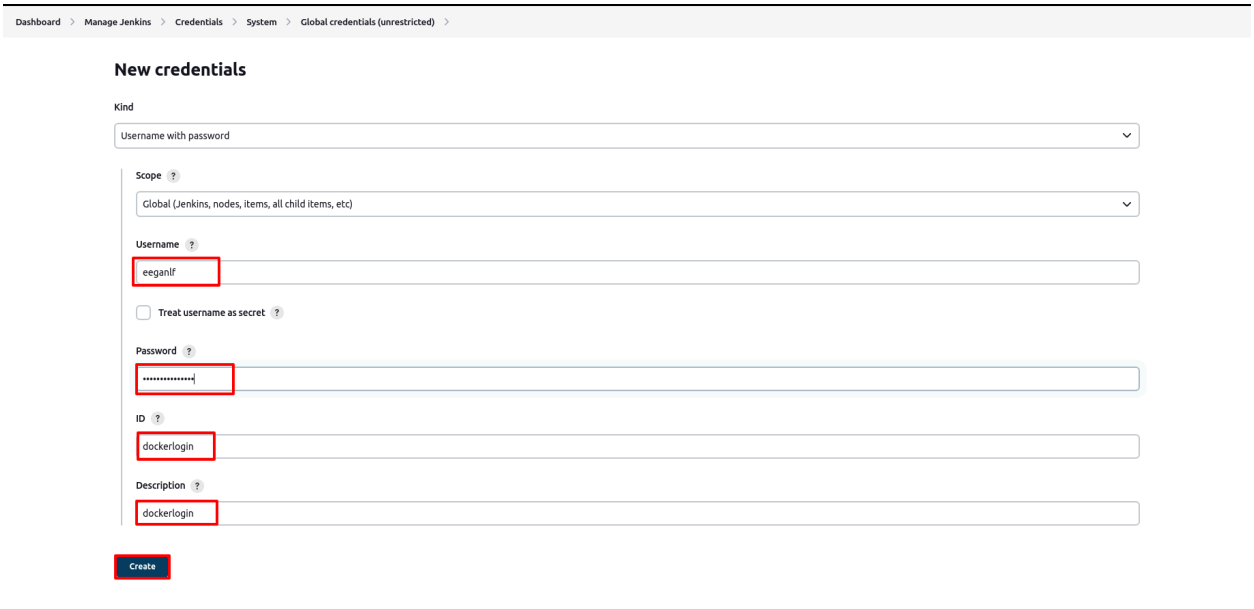
Add the username/password credentials to Jenkins with name **`dockerlogin`**. To do so, browse to **Jenkins Dashboard > Manage Jenkins > Credentials > Jenkins > Global Credentials > Add Credentials > Username and password**. Ensure you add the Docker Hub login and password with the ID as **`dockerlogin`**.

Reference the screenshots below to follow along.









Commit the changes you have made to the Jenkinsfile so that a new pipeline run is triggered. You will see the pipeline fail with an error similar to the one in the screenshot below.

```
$ docker exec --env ******** --env ******** --env ******** --env ******** --env ******** --env ******** --
env ******** --env ******** --env ******** --env ******** --env ******** --env ******** --env ******** --
env ******** --env ******** --env ******** --env ******** --env ******** --env ******** --env ******** --
env ******** --env ******** --env ******** --env ******** --env ******** --env ******** --env ******** --
env ******** --env ******** --env ******** --env ******** --env ******** --env ******** --env ******** --
env ******** --env ******** 30fdaceaf998f05ff55e943dc14873b7eca2f9f72c7018f6753627274a28edbe docker login
-u eeganlf -p ******** https://index.docker.io/v1/
OCI runtime exec failed: exec failed: unable to start container process: exec: "docker": executable file
not found in $PATH: unknown
```

In order to fix this error, you will have to add *Per Stage Agent* configurations.

Jenkinsfile Per Stage Agent Configurations

The following code contains the complete Jenkinsfile along with:

- Per Stage Agent configuration
- Conditional execution of the Docker **`package`** stage on the master
- Publishing the image on Docker Hub with the branch name as the tag

```
pipeline {

  agent none

  stages{
      stage("build"){
        when{
            changeset "**/worker/**"
        }

        agent{
          docker{
            image 'maven:3.9.8-sapmachine-21'
            args '-v $HOME/.m2:/root/.m2'
          }
        }

        steps{
          echo 'Compiling worker app..'
          dir('worker'){
            sh 'mvn compile'
          }
        }
      }
      stage("test"){
        when{
          changeset "**/worker/**"
        }
        agent{
          docker{
            image 'maven:3.9.8-sapmachine-21'
            args '-v $HOME/.m2:/root/.m2'
          }
        }
        steps{
```

```
          echo 'Running Unit Tests on worker app..'
          dir('worker'){
            sh 'mvn clean test'
           }

          }

      }
      stage("package"){
        when{
          branch 'master'
          changeset "**/worker/**"
        }
        agent{
          docker{
            image 'maven:3.9.8-sapmachine-21'
            args '-v $HOME/.m2:/root/.m2'
          }
        }
        steps{
          echo 'Packaging worker app'
          dir('worker'){
            sh 'mvn package -DskipTests'
            archiveArtifacts artifacts: '**/target/*.jar',
fingerprint: true
          }
        }
      }

      stage('docker-package'){
          agent any
          when{
            changeset "**/worker/**"
            branch 'master'
          }
          steps{
            echo 'Packaging worker app with docker'
            script{
              docker.withRegistry('https://index.docker.io/v1/',
'dockerlogin') {
                  def workerImage =
docker.build("xxxxx/worker:v${env.BUILD_ID}", "./worker")
                  workerImage.push()
                  workerImage.push("${env.BRANCH_NAME}")
                  workerImage.push("latest")
              }
```

```
            }
          }
        }
    }

    post{
      always{
          echo 'Building multibranch pipeline for worker is completed..'
      }
    }
}
```

You can access the **worker/Jenkinsfile** with the stage agents as well as conditional execution code here.

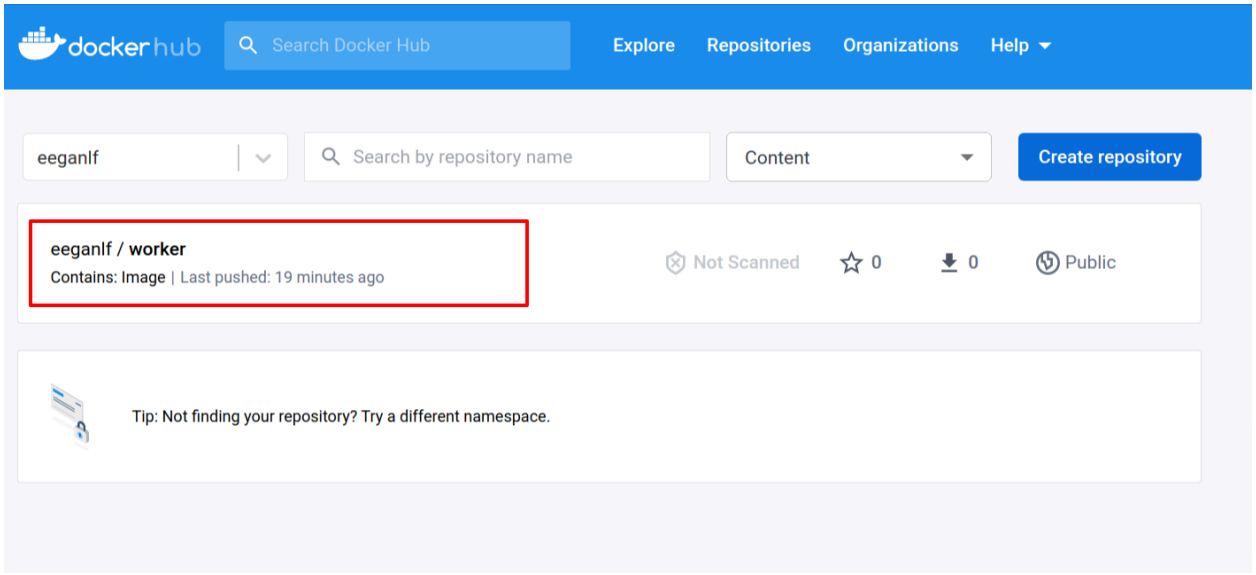
Once you edit your Jenkinsfile to this effect, commit the changes:

```
git commit -am "per stage agents, conditional execution"
```

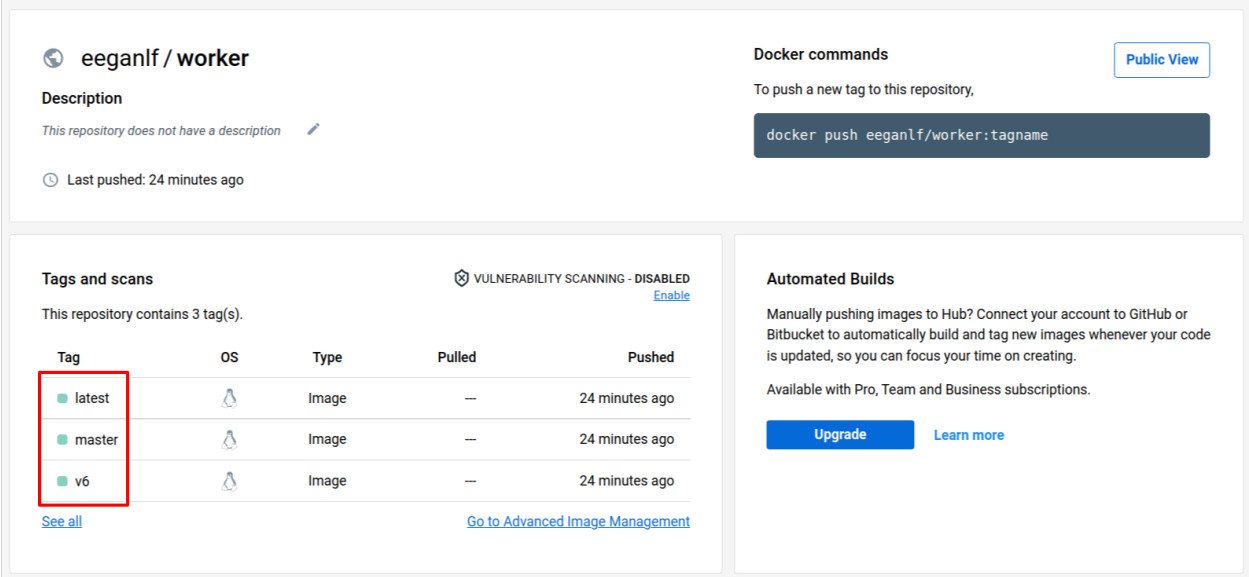
```
git push origin feature/dockerfiles
```

Merge the code into the master via a pull request. Remember not to delete the branch yet, as you will iterate the Jenkinsfile for **vote** and **result** applications with similar configurations.

Head over to the Docker Hub repository to validate that Jenkins has built and published images.



Click on the image.



Exercise: Vote and Result Application Dockerfiles

Follow the same process for **`vote`** and **`result`** applications:

- Check if Dockerfiles exist for **`vote`** and **`result`** applications. If not, create and check into the Git repository.
- Update Jenkinsfiles for **`vote`** and **`result`** applications with Docker Build and Publish stages.
- Make sure you refactor all Jenkinsfiles with Per Stage Docker Agent configurations.
- Validate the pipeline runs and images are visible on Docker Hub.

Result application - inside **`example-vote-app/result`**

The manual image sans Dockerfile creation process for the **`result`** application is as follows:

```
docker container run -idt --name nodebuild -p 80:80 node:22.4.0-slim

docker cp . nodebuild:/app  -this copies everything in . into
nodebuild container in the /app dir
```

```
docker exec -it nodebuild sh

which node

which npm

cd app

npm install  -this will install modules from package.json

npm audit fix  -this will fix known vulnerabilities in the modules
installed by npm

npm ls  - lists all modules downloaded by npm

npm test - runs unit tests to verify your code is ok

npm start
```

`waiting for db` - will be repeatedly displayed

```
ctrl + c

exit
```

`docker container commit nodebuild xxxxx/result:v1` - this will create an image and tag it.

`docker image history xxxxx/result:v1` - this will show a list of commands entered inside containers run with the image. These will help build a Dockerfile for the image to automate the creation of containers based on the image.

```
docker image push xxxxx/result:v1
```

The automated build, using Dockerfile, is as follows. Create the `result/Dockerfile` file with the following code:

```
FROM node:22.4.0-slim

# add curl for healthcheck
RUN apt-get update && \
    apt-get install -y --no-install-recommends curl tini && \
    rm -rf /var/lib/apt/lists/*

WORKDIR /usr/local/app

# have nodemon available for local dev use (file watching)
RUN npm install -g nodemon

COPY package*.json ./
```

```
RUN npm ci && \
 npm cache clean --force && \
 mv /usr/local/app/node_modules /node_modules

COPY . .

ENV PORT 80
EXPOSE 80

ENTRYPOINT ["/usr/bin/tini", "--"]
CMD ["node", "server.js"]
```

`docker image build -t xxxxx/result:v2 .` - DO NOT FORGET THE PERIOD because it tells Docker the path to the Dockerfile that will be used to build the image.

```
docker image tag xxxxx/result:v2 xxxxx/result:latest

docker image push xxxxx/result
```

Vote application - inside `example-vote-app/vote`

Create a `Dockerfile` inside the `vote/` directory with the following code:

```
# Define a base stage that uses the official python runtime base image
FROM python:3.11-slim AS base

# Add curl for healthcheck
RUN apt-get update && \
    apt-get install -y --no-install-recommends curl && \
    rm -rf /var/lib/apt/lists/*

# Set the application directory
WORKDIR /usr/local/app

# Install our requirements.txt
COPY requirements.txt ./requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

# Define a stage specifically for development, where it'll watch for
# filesystem changes
FROM base AS dev
RUN pip install watchdog
ENV FLASK_ENV=development
CMD ["python", "app.py"]

# Define the final stage that will bundle the application for
production
```

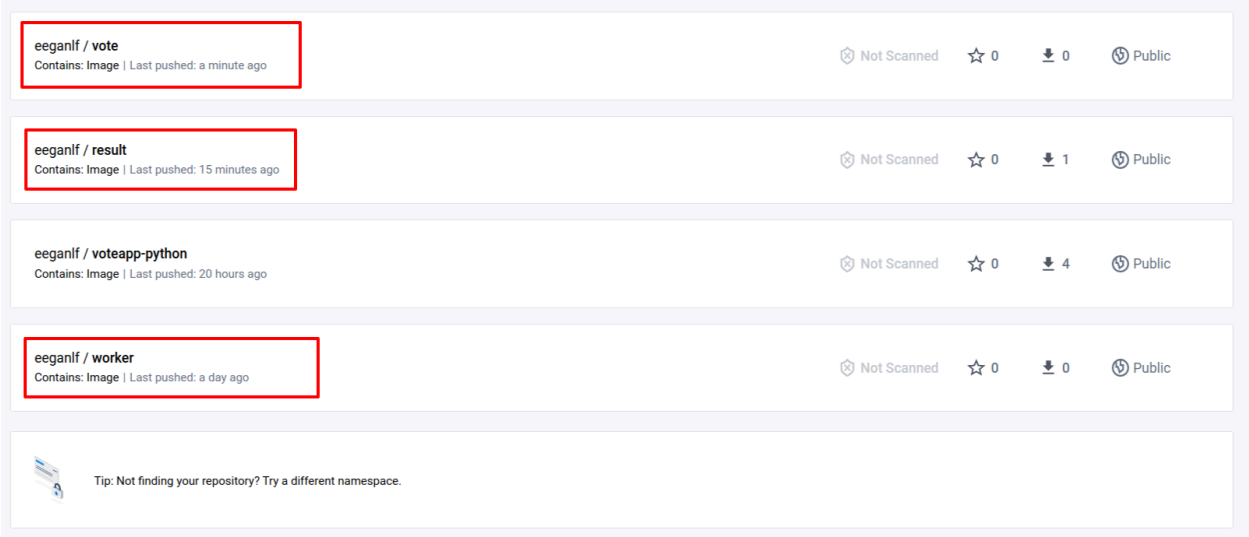
```
FROM base AS final

# Copy our code from the current folder to the working directory
inside the container
COPY . .

# Make port 80 available for links and/or publish
EXPOSE 80

# Define our command to be run when launching the container
CMD ["gunicorn", "app:app", "-b", "0.0.0.0:80", "--log-file", "-",
"--access-logfile", "-", "--workers", "4", "--keep-alive", "0"]
```

`docker image build -t xxxxx/vote:v1 .` -DO NOT FORGET THE PERIOD because it tells Docker the path to the Dockerfile that will be used to build the image.

`docker image tag xxxxx/vote:v1 xxxxx/vote:latest`

`docker image push xxxxx/vote`

You should now see your images on your Docker Hub repository page with any other images you may have pushed in the past:



These images will be required for the next lab.